load package

In [3]:
```
pip install xgboost
```

```
Collecting xgboost
  Downloading xgboost-1.6.1-py3-none-macosx_10_15_x86_64.macosx_11_0_x86_
64.macosx_12_0_x86_64.whl (1.7 MB)
     |[redacted]| 1.7 MB 11.3 MB/s
Requirement already satisfied: scipy in /Users/yuzhang/opt/anaconda3/lib/
python3.8/site-packages (from xgboost) (1.6.2)
Requirement already satisfied: numpy in /Users/yuzhang/opt/anaconda3/lib/
python3.8/site-packages (from xgboost) (1.19.5)
Installing collected packages: xgboost
Successfully installed xgboost-1.6.1
WARNING: You are using pip version 21.3.1; however, version 22.1.2 is ava
ilable.
You should consider upgrading via the '/Users/yuzhang/opt/anaconda3/bin/p
ython -m pip install --upgrade pip' command.
Note: you may need to restart the kernel to use updated packages.
```

In [4]:
```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler # data normalization
from sklearn.model_selection import train_test_split # data split
from sklearn.tree import DecisionTreeClassifier # Decision tree algorithm
from sklearn.neighbors import KNeighborsClassifier # KNN algorithm
from xgboost import XGBClassifier # XGBoost algorithm
from sklearn.linear_model import LogisticRegression # LogisticRegression
import xgboost as xgb

from sklearn.model_selection import cross_val_score
from sklearn.metrics import roc_auc_score,recall_score, precision_score, f1
from sklearn.metrics import accuracy_score, confusion_matrix, classificatio
from sklearn.metrics import r2_score
from sklearn.metrics import log_loss
from sklearn import metrics

import pandas as pd # data processing
import numpy as np # working with arrays
import matplotlib.pyplot as plt # visualization
import seaborn as sns
from termcolor import colored as cl # text customization
import itertools # advanced tools
```

Load data

In [5]:
```
train_df = pd.read_csv('/Users/yuzhang/Dropbox/Academia/UCI Lecturer/I&C_SC
test_df = pd.read_csv('/Users/yuzhang/Dropbox/Academia/UCI Lecturer/I&C_SCI
```

In [6]:
```
print('train_df')
train_df.head()
train_df.info()
train_df.shape
```

```
train_df
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 64384 entries, 0 to 64383
Columns: 114 entries, hospital_death to h_wbc_max
dtypes: float64(100), int64(7), object(7)
memory usage: 56.0+ MB
```

Out[6]:
```
(64384, 114)
```

In [5]:
```
print('test_df')
train_df.head()
train_df.info()
train_df.shape
```

```
test_df
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 64384 entries, 0 to 64383
Columns: 114 entries, hospital_death to h_wbc_max
dtypes: float64(100), int64(7), object(7)
memory usage: 56.0+ MB
```

Out[5]:
```
(64384, 114)
```

For supervised learning classification problem target variable is hospital_death.

In [6]:
```
train_df['hospital_death'].dtype
```

Out[6]:
```
dtype('int64')
```

Encode Categoricals variables

In [7]:
```
s = (train_df.dtypes == 'object')
object_cols = list(s[s].index)

print("Categorical variables:")
print(object_cols)
```

```
Categorical variables:
['ethnicity', 'gender', 'hospital_admit_source', 'icu_admit_source', 'icu
_stay_type', 'icu_type', 'bodysystem_diagnosis_admit']
```

In [8]:
```
cat_train_df = train_df.select_dtypes(include='object')
cat_train_df.head()
cat_train_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 64384 entries, 0 to 64383
Data columns (total 7 columns):
 #   Column                      Non-Null Count  Dtype 
---  ------                      --------------  ----- 
 0   ethnicity                   63118 non-null  object
 1   gender                      64369 non-null  object
 2   hospital_admit_source       48431 non-null  object
 3   icu_admit_source            64303 non-null  object
 4   icu_stay_type               64384 non-null  object
 5   icu_type                    64384 non-null  object
 6   bodysystem_diagnosis_admit  63234 non-null  object
dtypes: object(7)
memory usage: 3.4+ MB
```

In [9]:
```
train_df = train_df.drop(['hospital_id','icu_id','readmission_status','ethn
'gender','hospital_admit_source','icu_admit_source','icu_stay_type','icu_ty
```

In [10]:
```
test_df = test_df.drop(['hospital_id','icu_id','readmission_status','ethnic
'gender','hospital_admit_source','icu_admit_source','icu_stay_type','icu_ty
```

In [11]:
```
train_df.shape
```

Out[11]:
```
(64384, 105)
```

In [12]:
```
test_df.shape
```

Out[12]:
```
(27329, 105)
```

In [13]:
```
def display_columns_uniqvals(df):
    for i, col in enumerate(df.columns.tolist()):
         print('\n ({} {}) Uniq: {} UniqSz: {}'.format(i,col, df[col].uniqu
    print('\n')
```

In [14]:
```
display_columns_uniqvals(train_df)
```

```
 221. 219. 222. 230.] UniqSz: 144


 (37 h_sysbp_max) Uniq: [118.  90. 122. 106. 112. 119. 127. 105. 144.  9
9. 135.  96. 150. 175.
 103.  98. 101. 124. 111. 171. 174.  nan 126. 108. 165. 168.  95. 115.
 163. 133. 129. 149. 151. 109. 145. 116. 147. 223. 136. 159. 146. 138.
 128. 123. 120. 160. 107. 139. 121. 153. 166. 173.  82. 156. 158. 137.
 148. 125. 193. 117. 143. 102. 169.  86.  94.  97. 177. 134. 140. 181.
 154. 100. 155. 110. 170. 196. 178. 184.  92.  89. 104.  88. 190. 161.
 152. 142. 130. 198. 113. 157.  91. 202.  75. 192. 172.  85. 114. 176.
 162. 167. 141.  84. 183. 182.  76. 164. 131. 132. 210. 189. 214. 207.
 191. 194. 179.  87. 180. 188.  93. 208. 200. 186. 195. 197. 201.  77.
 206.  80. 218. 199.  79.  78. 187. 203. 205. 204.  81. 185.  83. 212.
 217. 220. 215. 209. 211. 222. 221. 219. 213. 216.] UniqSz: 150

 (38 d_sysbp_noninvasive_max) Uniq: [140. 126. 104. 106. 124. 125. 137. 1
35. 156. 190. 172. 115. 150. 144.
 175. 132. 146. 117. 171.  nan 105. 110. 143. 131. 127. 101. 178. 196.
 128. 151. 163. 159. 136. 129. 149. 164. 141. 122. 173. 134. 232. 139.
 147. 120. 203. 168. 183. 138. 152. 116. 133. 228. 153. 166. 198. 118.
```

In [15]:
```
## split data

from sklearn.model_selection import train_test_split

train = train_df.copy()

y = train['hospital_death']

X = train.drop(['hospital_death'], axis=1).select_dtypes(exclude=['object']

X_train, X_valid, y_train, y_valid = train_test_split(X, y, train_size=0.8,
```

In [16]:
```
X.shape
```

Out[16]: (64384, 103)

In [17]:
```
X_valid.shape
```

Out[17]: (12877, 103)

In [18]:
```
X_train.shape
```

Out[18]: (51507, 103)

In [19]:
```
test = test_df.drop(labels='hospital_death', axis=1).copy()

# To keep things simple, we'll use only numerical predictors

X_test = test.select_dtypes(exclude=['object'])

X_test.shape
X_test.head()
```

Out[19]:

	patient_id	d_diasbp_min	h_diasbp_min	d_diasbp_noninvasive_min	h_diasbp_noninvasive_min	d_h
0	2	68.0	76.0	68.0	76.0	
1	4	31.0	37.0	31.0	37.0	
2	5	44.0	62.0	44.0	62.0	
3	7	59.0	73.0	59.0	73.0	
4	12	48.0	63.0	48.0	63.0	

5 rows × 103 columns

Imputatoin

In [20]:
```
##from sklearn.impute import SimpleImputer

my_imputer = SimpleImputer()
imputed_X_train = pd.DataFrame(my_imputer.fit_transform(X_train))
imputed_X_valid = pd.DataFrame(my_imputer.transform(X_valid))

imputed_X_train.columns = X_train.columns
imputed_X_valid.columns = X_valid.columns
```

In [21]:
```
imputed_X_train.shape
```

Out[21]:
```
(51507, 103)
```

In [22]:
```
y_train.shape
```

Out[22]:
```
(51507,)
```

In [23]:
```
my_imputer = SimpleImputer()
imputed_X_test = pd.DataFrame(my_imputer.fit_transform(X_test))


# Imputation removed column names; put them back
imputed_X_test.columns = X_test.columns
```

In [24]:
```
imputed_X_test.shape
```

Out[24]:
```
(27329, 103)
```

In [25]:
```
X_test.shape
```

Out[25]:
```
(27329, 103)
```

Machine Learning

In [28]:
```
#1. Decision Tree
from sklearn.tree import DecisionTreeClassifier
from imblearn.over_sampling import SMOTE
smo = SMOTE()

X_train = imputed_X_train
X_valid = imputed_X_valid

X_smo, y_smo = smo.fit_resample(X_train, y_train)

tree_model = DecisionTreeClassifier(max_depth=4,splitter="best",min_samples
                                    min_samples_leaf=4,max_features=None,cl
tree_model.fit(X_smo, y_smo)
tree_yhat = tree_model.predict(X_valid)
```

In [29]:
```
# 2. K-Nearest Neighbors
from sklearn.neighbors import KNeighborsClassifier # KNN algorithm
from imblearn.over_sampling import SMOTE
smo = SMOTE()

X_train = imputed_X_train
X_valid = imputed_X_valid

X_smo, y_smo = smo.fit_resample(X_train, y_train)

knn = KNeighborsClassifier(n_neighbors=4, weights="distance",algorithm="kd_
knn.fit(X_smo, y_smo )
knn_yhat = knn.predict(X_valid)
```

In [30]:
```
# 3. LR
from imblearn.over_sampling import SMOTE
from sklearn.linear_model import LogisticRegression

smo = SMOTE()
X_train = imputed_X_train
X_valid = imputed_X_valid

lr = LogisticRegression(tol=1e-4)
lr.fit(X_smo, y_smo )

X_smo, y_smo = smo.fit_resample(X_train, y_train)
lr_yhat = lr.predict(X_valid)
```

```
C:\ProgramData\Anaconda3\lib\site-packages\sklearn\linear_model\_logisti
c.py:763: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown i
n:
    https://scikit-learn.org/stable/modules/preprocessing.html (https://s
cikit-learn.org/stable/modules/preprocessing.html)
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-re
gression (https://scikit-learn.org/stable/modules/linear_model.html#logis
tic-regression)
  n_iter_i = _check_optimize_result(
```

In [31]:
```
# 4.  XGboosting
from xgboost import XGBClassifier # XGBoost algorithm
import warnings
warnings.filterwarnings('ignore')
from imblearn.over_sampling import SMOTE
smo = SMOTE()

X_train = imputed_X_train
X_valid = imputed_X_valid

X_smo, y_smo = smo.fit_resample(X_train, y_train)

xgb = XGBClassifier(max_depth=7,scale_pos_weight=3,n_estimators=200,subsamp
                        colsample_bytree=0.9,learning_rate=0.25,alpha=1.5,r

xgb.fit(X_smo, y_smo )
xgb_yhat = xgb.predict(X_valid)
```

```
[00:57:05] WARNING: C:/Users/Administrator/workspace/xgboost-win64_releas
e_1.3.0/src/learner.cc:541:
Parameters: { class_weight } might not be used.

  This may not be accurate due to some parameters are only used in langua
ge bindings but
  passed down to XGBoost core.  Or some parameters are not used but slip
through this
  verification. Please open an issue if you find above cases.


[00:57:06] WARNING: C:/Users/Administrator/workspace/xgboost-win64_releas
e_1.3.0/src/learner.cc:1061: Starting in XGBoost 1.3.0, the default evalu
ation metric used with the objective 'binary:logistic' was changed from
'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore
the old behavior.
```

Model Evaluation

In [32]:
```
## K-folds cross-validation
from sklearn.metrics import accuracy_score
```

In [33]:
```
# Perform cross-validation

scores = cross_val_score(tree_model, X_train, y_train, scoring='accuracy',
# report performance
mean_score=np.mean(np.asarray(scores))
print(mean_score)
```

```
0.7989008385925706
```

In [34]:
```
# Perform cross-validation

scores = cross_val_score(knn, X_train, y_train, scoring='accuracy', cv=5)
# report performance
mean_score=np.mean(np.asarray(scores))
print(mean_score)
```

```
0.9126138053368601
```

In [35]:
```
# Perform cross-validation

scores = cross_val_score(lr, X_train, y_train, scoring='accuracy', cv=5)
# report performance
mean_score=np.mean(np.asarray(scores))
print(mean_score)
```

```
0.9205544898214303
```

In [36]:
```
# Perform cross-validation

scores = cross_val_score(xgb, X_train, y_train, scoring='accuracy', cv=5)
# report performance
mean_score=np.mean(np.asarray(scores))
print(mean_score)
```

```
[00:59:00] WARNING: C:/Users/Administrator/workspace/xgboost-win64_releas
e_1.3.0/src/learner.cc:541:
Parameters: { class_weight } might not be used.

  This may not be accurate due to some parameters are only used in langua
ge bindings but
  passed down to XGBoost core.  Or some parameters are not used but slip
through this
  verification. Please open an issue if you find above cases.


[00:59:00] WARNING: C:/Users/Administrator/workspace/xgboost-win64_releas
e_1.3.0/src/learner.cc:1061: Starting in XGBoost 1.3.0, the default evalu
ation metric used with the objective 'binary:logistic' was changed from
'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore
the old behavior.
[00:59:23] WARNING: C:/Users/Administrator/workspace/xgboost-win64_releas
e_1.3.0/src/learner.cc:541:
Parameters: { class_weight } might not be used.

  This may not be accurate due to some parameters are only used in langua
ge bindings but
  passed down to XGBoost core.  Or some parameters are not used but slip
through this
  verification. Please open an issue if you find above cases.


[00:59:23] WARNING: C:/Users/Administrator/workspace/xgboost-win64_releas
e_1.3.0/src/learner.cc:1061: Starting in XGBoost 1.3.0, the default evalu
ation metric used with the objective 'binary:logistic' was changed from
'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore
the old behavior.
[00:59:44] WARNING: C:/Users/Administrator/workspace/xgboost-win64_releas
e_1.3.0/src/learner.cc:541:
Parameters: { class_weight } might not be used.

  This may not be accurate due to some parameters are only used in langua
ge bindings but
  passed down to XGBoost core.  Or some parameters are not used but slip
through this
  verification. Please open an issue if you find above cases.


[00:59:45] WARNING: C:/Users/Administrator/workspace/xgboost-win64_releas
e_1.3.0/src/learner.cc:1061: Starting in XGBoost 1.3.0, the default evalu
ation metric used with the objective 'binary:logistic' was changed from
'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore
the old behavior.
[01:00:07] WARNING: C:/Users/Administrator/workspace/xgboost-win64_releas
e_1.3.0/src/learner.cc:541:
```

```
Parameters: { class_weight } might not be used.

  This may not be accurate due to some parameters are only used in langua
ge bindings but
  passed down to XGBoost core.  Or some parameters are not used but slip
through this
  verification. Please open an issue if you find above cases.


[01:00:07] WARNING: C:/Users/Administrator/workspace/xgboost-win64_releas
e_1.3.0/src/learner.cc:1061: Starting in XGBoost 1.3.0, the default evalu
ation metric used with the objective 'binary:logistic' was changed from
'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore
the old behavior.
[01:00:30] WARNING: C:/Users/Administrator/workspace/xgboost-win64_releas
e_1.3.0/src/learner.cc:541:
Parameters: { class_weight } might not be used.

  This may not be accurate due to some parameters are only used in langua
ge bindings but
  passed down to XGBoost core.  Or some parameters are not used but slip
through this
  verification. Please open an issue if you find above cases.


[01:00:31] WARNING: C:/Users/Administrator/workspace/xgboost-win64_releas
e_1.3.0/src/learner.cc:1061: Starting in XGBoost 1.3.0, the default evalu
ation metric used with the objective 'binary:logistic' was changed from
'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore
the old behavior.
0.9281844739691337
```

Classification report

In [37]:
```python
print(metrics.classification_report(y_valid, tree_yhat))
```

```
              precision    recall  f1-score   support

           0       0.96      0.86      0.91     11821
           1       0.28      0.63      0.39      1056

    accuracy                           0.84     12877
   macro avg       0.62      0.74      0.65     12877
weighted avg       0.91      0.84      0.86     12877
```

In [38]:
```
print(metrics.classification_report(y_valid, knn_yhat))
```

```
              precision    recall  f1-score   support

           0       0.93      0.82      0.87     11821
           1       0.14      0.32      0.20      1056

    accuracy                           0.78     12877
   macro avg       0.54      0.57      0.53     12877
weighted avg       0.87      0.78      0.82     12877
```

In [39]:
```
print(metrics.classification_report(y_valid, lr_yhat))
```

```
              precision    recall  f1-score   support

           0       0.96      0.74      0.83     11821
           1       0.19      0.70      0.30      1056

    accuracy                           0.73     12877
   macro avg       0.58      0.72      0.57     12877
weighted avg       0.90      0.73      0.79     12877
```

In [40]:
```
print(metrics.classification_report(y_valid, xgb_yhat))
```

```
              precision    recall  f1-score   support

           0       0.95      0.97      0.96     11821
           1       0.53      0.43      0.47      1056

    accuracy                           0.92     12877
   macro avg       0.74      0.70      0.71     12877
weighted avg       0.92      0.92      0.92     12877
```

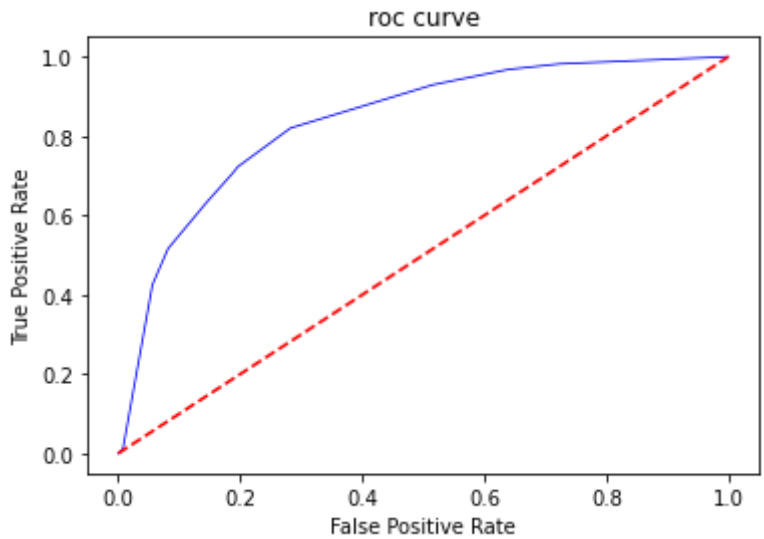
ROC AUC curve

In [41]:
```
def plot_roc_curve(y_true, y_proba):
    from sklearn.metrics import roc_curve
    import matplotlib.pyplot as plt
    fpr,tpr,threshold = roc_curve(y_true,y_proba, pos_label=1)
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title('roc curve')
    plt.plot(fpr, tpr, color = 'b', linewidth = 0.8)
    plt.plot([0,1],[0,1],'r--')
```

In [42]:
```
y_predict = tree_model.predict_proba(imputed_X_valid)[:, 1]
roc_auc_score(y_valid, y_predict)
```

Out[42]: 0.8375477129812634

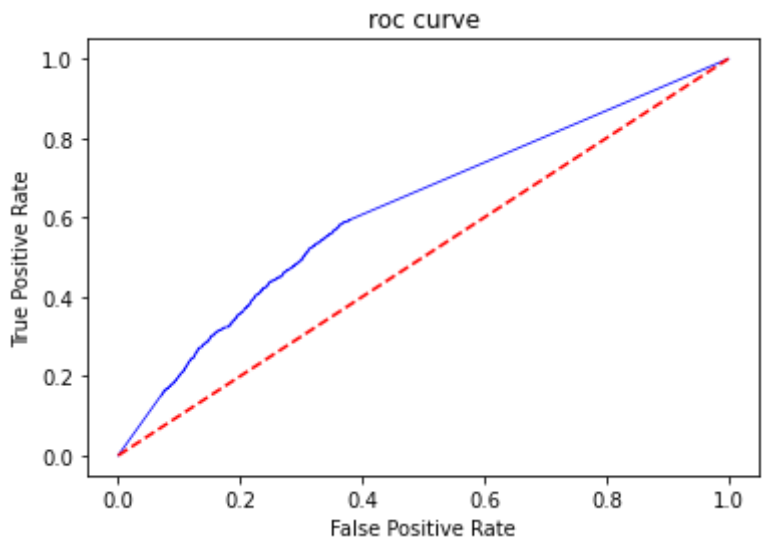
In [43]:
```
plot_roc_curve(y_valid, y_predict)
```



In [44]:
```
y_predict = knn.predict_proba(imputed_X_valid)[:, 1]
roc_auc_score(y_valid, y_predict)
```

Out[44]: 0.6205668824485444

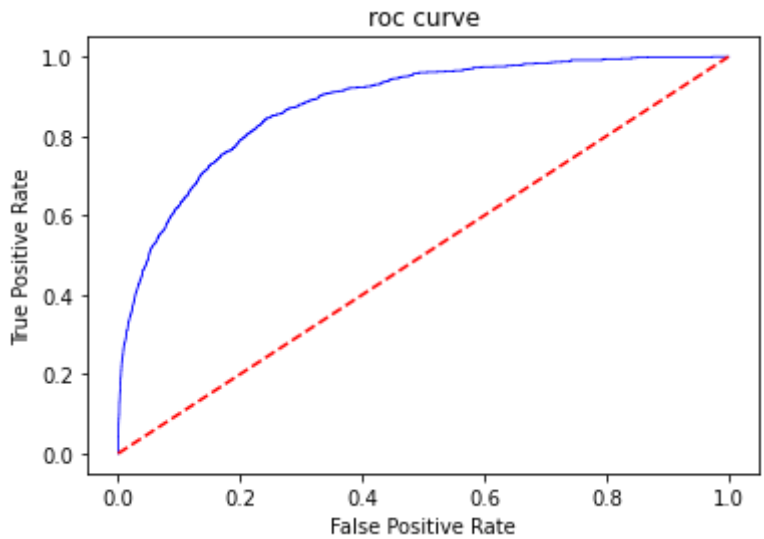
In [45]:
```
plot_roc_curve(y_valid, y_predict)
```



In [46]:
```
y_predict = xgb.predict_proba(imputed_X_valid)[:, 1]
roc_auc_score(y_valid, y_predict)
```

Out[46]: 0.8798756802864958

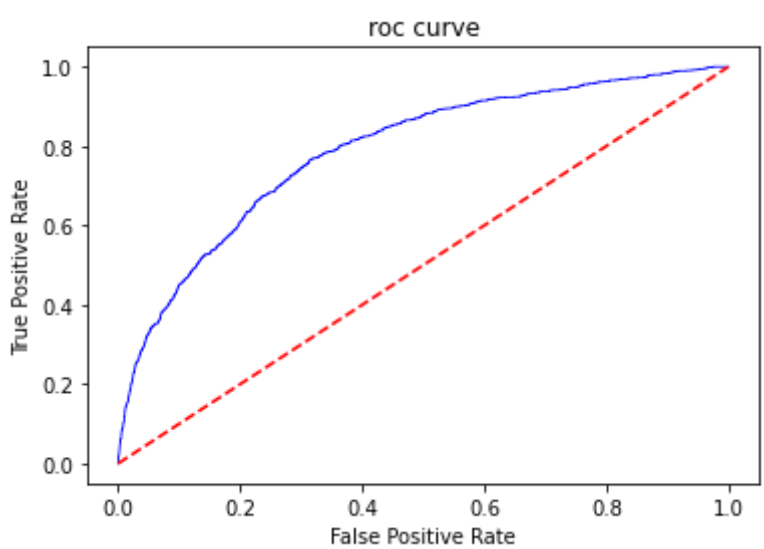
In [47]:
```
plot_roc_curve(y_valid, y_predict)
```



In [48]:
```
y_predict = lr.predict_proba(imputed_X_valid)[:, 1]
roc_auc_score(y_valid, y_predict)
```

Out[48]: 0.7884322616658078

In [49]:
```
plot_roc_curve(y_valid, y_predict)
```



Accuracy score Accuracy score = No.of correct predictions / Total no.of predictions

In [39]:
```
print(cl('ACCURACY SCORE', attrs = ['bold']))
print(cl('----------------------------------------------------------------
print(cl('Accuracy score of the Decision Tree model is {}'.format(accuracy_
print(cl('----------------------------------------------------------------
print(cl('Accuracy score of the KNN model is {}'.format(accuracy_score(y_va
print(cl('----------------------------------------------------------------
print(cl('Accuracy score of the XGBoost model is {}'.format(accuracy_score(
print(cl('----------------------------------------------------------------
print(cl('Accuracy score of the LogisticRegression model is {}'.format(accu
print(cl('----------------------------------------------------------------

print(cl('RECALL SCORE', attrs = ['bold']))
print(cl('----------------------------------------------------------------
print(cl('Recall score of the Decision Tree model is {}'.format(recall_scor
print(cl('----------------------------------------------------------------
print(cl('Recall score of the KNN model is {}'.format(recall_score(y_valid,
print(cl('----------------------------------------------------------------
print(cl('Recall score of the XGBoost model is {}'.format(recall_score(y_va
print(cl('----------------------------------------------------------------
print(cl('Recall score of the LogisticRegression model is {}'.format(recall
print(cl('----------------------------------------------------------------
```

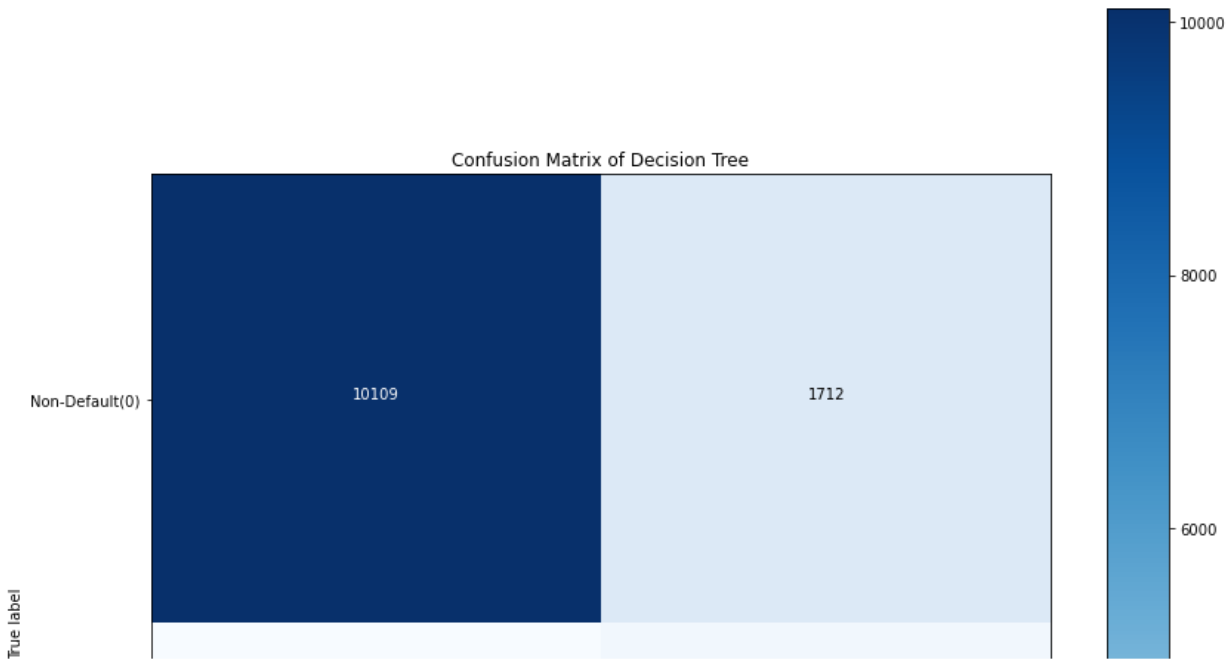
```
ACCURACY SCORE
-----------------------------------------------------------------------
Accuracy score of the Decision Tree model is 0.8529160518754368
-----------------------------------------------------------------------
Accuracy score of the KNN model is 0.7817814708394812
-----------------------------------------------------------------------
Accuracy score of the XGBoost model is 0.9224974761202144
-----------------------------------------------------------------------
Accuracy score of the LogisticRegression model is 0.7131319406694105
-----------------------------------------------------------------------
RECALL SCORE
-----------------------------------------------------------------------
Recall score of the Decision Tree model is 0.5416666666666666
-----------------------------------------------------------------------
Recall score of the KNN model is 0.33049242424242425
-----------------------------------------------------------------------
Recall score of the XGBoost model is 0.41382575757575757
-----------------------------------------------------------------------
Recall score of the LogisticRegression model is 0.7054924242424242
-----------------------------------------------------------------------
```

F1 Score F1 score = 2((precision * recall) / (precision + recall))

Confusion Matrix

In [50]:
```
#Confusion Matrix

# defining the plot function

def plot_confusion_matrix(cm, classes, title, normalize = False, cmap = plt
    title = 'Confusion Matrix of {}'.format(title)
    if normalize:
        cm = cm.astype(float) / cm.sum(axis=1)[:, np.newaxis]

    plt.figure(figsize=(6*2, 6*2))    # 8, 6
    plt.imshow(cm, interpolation='nearest', cmap=cmap)
    plt.title(title)
    plt.colorbar()
    tick_marks = np.arange(len(classes))
    plt.xticks(tick_marks, classes, rotation = 45)
    plt.yticks(tick_marks, classes)

    fmt = '.2f' if normalize else 'd'
    thresh = cm.max() / 2.
    for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
        plt.text(j, i, format(cm[i, j], fmt),
                 horizontalalignment = 'center',
                 color = 'white' if cm[i, j] > thresh else 'black')

    plt.tight_layout()
    plt.ylabel('True label')
    plt.xlabel('Predicted label')

# Compute confusion matrix for the models

tree_matrix = confusion_matrix(y_valid, tree_yhat, labels = [0, 1]) # Decis
knn_matrix = confusion_matrix(y_valid, knn_yhat, labels = [0, 1]) # K-Neare
xgb_matrix = confusion_matrix(y_valid, xgb_yhat, labels = [0, 1]) # XGBoost
lr_matrix = confusion_matrix(y_valid, lr_yhat, labels = [0, 1]) # Logistic

# Plot the confusion matrix
plt.rcParams['figure.figsize'] = (6, 6)

# 1. Decision tree

tree_cm_plot = plot_confusion_matrix(tree_matrix,
                                     classes = ['Non-Default(0)','Default(1)'],
                                     normalize = False, title = 'Decision Tree')
plt.savefig('tree_cm_plot.png')
plt.show()

# 2. K-Nearest Neighbors

knn_cm_plot = plot_confusion_matrix(knn_matrix,
                                     classes = ['Non-Default(0)','Default(1)'],
                                     normalize = False, title = 'KNN')
plt.savefig('knn_cm_plot.png')
plt.show()

# 3. XGBoost
```

```
xgb_cm_plot = plot_confusion_matrix(xgb_matrix,
                                    classes = ['Non-Default(0)','Default(1)'],
                                    normalize = False, title = 'XGBoost')
plt.savefig('xgb_cm_plot.png')


# 4. Logistic Regression

lr_cm_plot = plot_confusion_matrix(lr_matrix,
                                    classes = ['Non-Default(0)','Default(1)'],
                                    normalize = False, title = 'LogisticRegress
plt.savefig('lr_cm_plot.png')
```



Feature Importance

Tuning the Tree

In [50]:
```
from sklearn.tree import DecisionTreeClassifier

tree_model = DecisionTreeClassifier(max_depth=3, random_state=1)
tree_model.fit((X_smo, y_smo)
```

Out[50]:
```
DecisionTreeClassifier(max_depth=3, random_state=1)
```

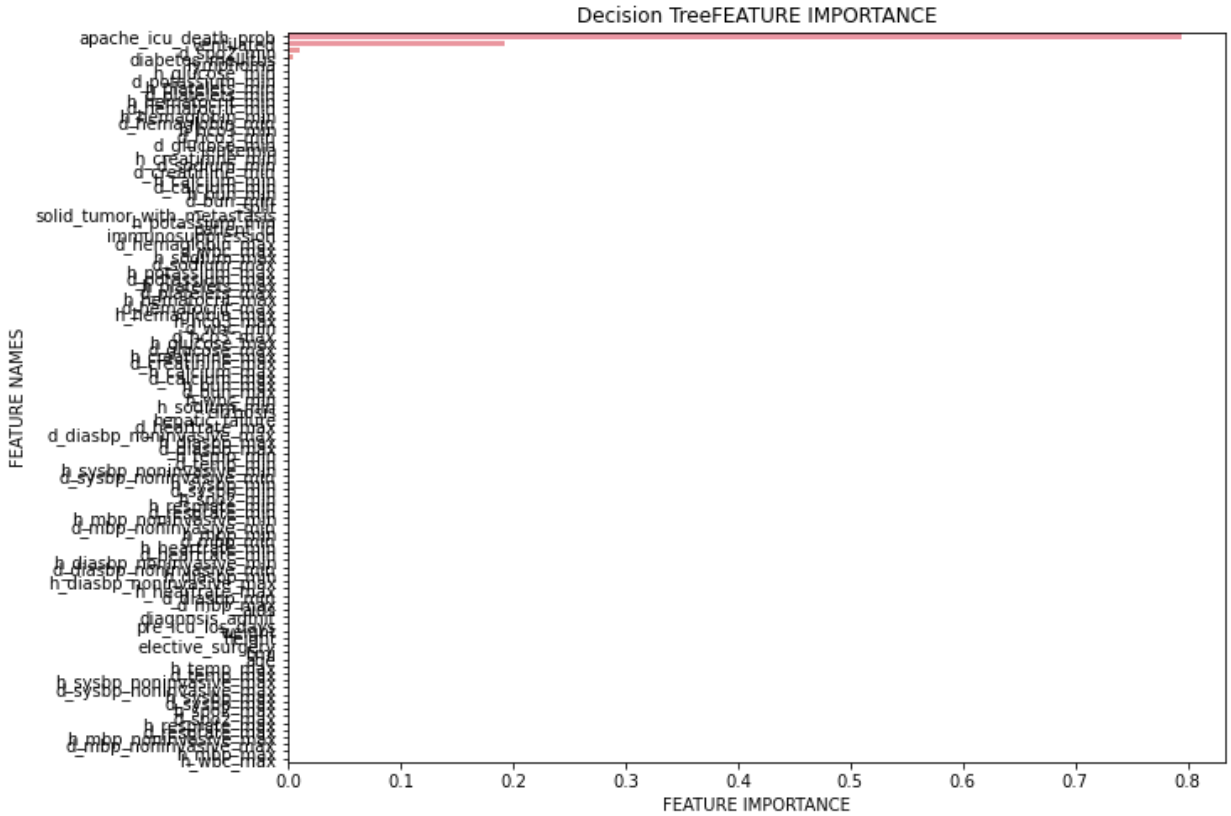
In [51]:
```
xgb = XGBClassifier()
xgb.fit((X_smo, y_smo)
```

```
[16:27:18] WARNING: C:/Users/Administrator/workspace/xgboost-win64_releas
e_1.3.0/src/learner.cc:1061: Starting in XGBoost 1.3.0, the default evalu
ation metric used with the objective 'binary:logistic' was changed from
'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore
the old behavior.
```

Out[51]:
```
XGBClassifier(base_score=0.5, booster='gbtree', colsample_bylevel=1,
              colsample_bynode=1, colsample_bytree=1, gamma=0, gpu_id=-1,
              importance_type='gain', interaction_constraints='',
              learning_rate=0.300000012, max_delta_step=0, max_depth=6,
              min_child_weight=1, missing=nan, monotone_constraints='()',
              n_estimators=100, n_jobs=8, num_parallel_tree=1, random_sta
te=0,
              reg_alpha=0, reg_lambda=1, scale_pos_weight=1, subsample=1,
              tree_method='exact', validate_parameters=1, verbosity=None)
```

In [51]:
```
def plot_feature_importance(importance,names,model_type):

    feature_importance = np.array(importance)
    feature_names = np.array(names)

    data={'feature_names':feature_names,'feature_importance':feature_import
    df = pd.DataFrame(data)

    df.sort_values(by=['feature_importance'], ascending=False,inplace=True)

    #Define size of bar plot
    plt.figure(figsize=(10,8))
    #Plot Searborn bar chart
    sns.barplot(x=df['feature_importance'], y=df['feature_names'])
    #Add chart labels
    plt.title(model_type + 'FEATURE IMPORTANCE')
    plt.xlabel('FEATURE IMPORTANCE')
    plt.ylabel('FEATURE NAMES')
```

In [52]:
```
plot_feature_importance(tree_model.feature_importances_,X.columns,'Decision
```

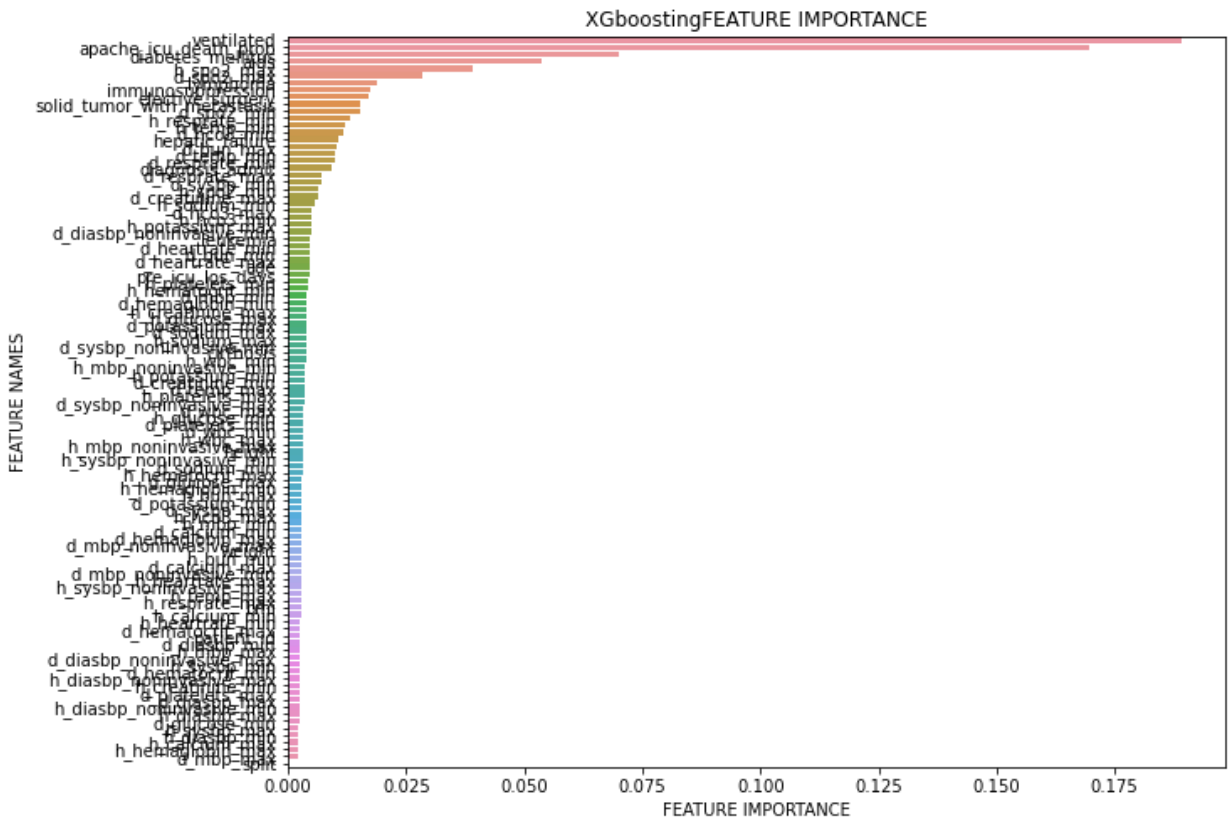


In [70]:
```
pd.DataFrame({'feature_names':X.columns,'feature_importance':tree_model.fea
```

Out[70]:

	feature_names	feature_importance
48	apache_icu_death_prob	0.785090
49	ventilated	0.191407
52	diabetes_mellitus	0.022786
15	d_sysbp_min	0.000717
13	d_spo2_min	0.000000

In [53]:
```
plot_feature_importance(xgb.feature_importances_,X.columns,'XGboosting')
```



In [72]:
```
pd.DataFrame({'feature_names':X.columns,'feature_importance':xgb.feature_im
```

Out[72]:

	feature_names	feature_importance
48	apache_icu_death_prob	0.785090
49	ventilated	0.191407
52	diabetes_mellitus	0.022786
15	d_sysbp_min	0.000717
13	d_spo2_min	0.000000

In [59]:
```
# Create a Graphviz file.
from sklearn.tree import export_graphviz
from sklearn import tree

export_graphviz(tree_model, out_file='tree.dot', feature_names=X.columns)
```

In [60]:
```
import pydotplus

tree = tree.export_graphviz(tree_model, out_file=None)
graph = pydotplus.graph_from_dot_data(tree)
graph.write_jpg("tree.jpg")
```

Out[60]:
```
True
```

only four features are important

select only four feature and re-run the data

In [63]:
```
features = ['apache_icu_death_prob','ventilated','diabetes_mellitus','d_sys
```

In [64]:
```
## split data

from sklearn.model_selection import train_test_split

train = train_df.copy()

y = train['hospital_death']

X = train.drop(['hospital_death'], axis=1)[features]

X_train, X_valid, y_train, y_valid = train_test_split(X, y, train_size=0.8,
```

In [65]:
```
X.shape
```

Out[65]:
```
(64384, 4)
```

In [66]:
```
X_valid.shape
```

Out[66]:
```
(12877, 4)
```

In [67]:
```
X_train.shape
```

Out[67]:
```
(51507, 4)
```

In [68]:
```
test = test_df.drop(labels='hospital_death', axis=1).copy()

# To keep things simple, we'll use only numerical predictors

X_test = test[features]

X_test.shape
X_test.head()
```

Out[68]:

	apache_icu_death_prob	ventilated	diabetes_mellitus	d_sysbp_min
0	0.01	0.0	0.0	98.0
1	0.11	1.0	0.0	72.0
2	0.16	0.0	0.0	121.0
3	0.01	0.0	0.0	97.0
4	0.10	0.0	0.0	87.0

Imputatoin

In [69]:
```
##from sklearn.impute import SimpleImputer

my_imputer = SimpleImputer()
imputed_X_train = pd.DataFrame(my_imputer.fit_transform(X_train))
imputed_X_valid = pd.DataFrame(my_imputer.transform(X_valid))

imputed_X_train.columns = X_train.columns
imputed_X_valid.columns = X_valid.columns
```

In [70]:
```
my_imputer = SimpleImputer()
imputed_X_test = pd.DataFrame(my_imputer.fit_transform(X_test))


# Imputation removed column names; put them back
imputed_X_test.columns = X_test.columns
```

In [71]:
```
X_test.shape
```

Out[71]: (27329, 4)

In [72]:
```
#1. Decision Tree
from sklearn.tree import DecisionTreeClassifier
from imblearn.over_sampling import SMOTE
smo = SMOTE()

X_train = imputed_X_train
X_valid = imputed_X_valid

X_smo, y_smo = smo.fit_resample(X_train, y_train)

tree_model = DecisionTreeClassifier(max_depth=4,splitter="best",min_samples
                                    min_samples_leaf=4,max_features=None,cl
tree_model.fit(X_smo, y_smo)
tree_yhat = tree_model.predict(X_valid)
```

In [73]:
```
# 2. K-Nearest Neighbors
from sklearn.neighbors import KNeighborsClassifier # KNN algorithm
from imblearn.over_sampling import SMOTE
smo = SMOTE()

X_train = imputed_X_train
X_valid = imputed_X_valid

X_smo, y_smo = smo.fit_resample(X_train, y_train)

knn = KNeighborsClassifier(n_neighbors=4, weights="distance",algorithm="kd_
knn.fit(X_smo, y_smo )
knn_yhat = knn.predict(X_valid)
```

In [74]:
```
# 3. LR
from imblearn.over_sampling import SMOTE
from sklearn.linear_model import LogisticRegression

smo = SMOTE()
X_train = imputed_X_train
X_valid = imputed_X_valid

lr = LogisticRegression(tol=1e-4)
lr.fit(X_smo, y_smo )

X_smo, y_smo = smo.fit_resample(X_train, y_train)
lr_yhat = lr.predict(X_valid)
```

In [75]:
```
# 4.  XGboosting
from xgboost import XGBClassifier # XGBoost algorithm
import warnings
warnings.filterwarnings('ignore')
from imblearn.over_sampling import SMOTE
smo = SMOTE()

X_train = imputed_X_train
X_valid = imputed_X_valid

X_smo, y_smo = smo.fit_resample(X_train, y_train)

xgb = XGBClassifier(max_depth=7,scale_pos_weight=3,n_estimators=200,subsamp
                        colsample_bytree=0.9,learning_rate=0.25,alpha=1.5,r

xgb.fit(X_smo, y_smo )
xgb_yhat = xgb.predict(X_valid)
```

```
[01:08:52] WARNING: C:/Users/Administrator/workspace/xgboost-win64_releas
e_1.3.0/src/learner.cc:541:
Parameters: { class_weight } might not be used.

  This may not be accurate due to some parameters are only used in langua
ge bindings but
  passed down to XGBoost core.  Or some parameters are not used but slip
through this
  verification. Please open an issue if you find above cases.


[01:08:52] WARNING: C:/Users/Administrator/workspace/xgboost-win64_releas
e_1.3.0/src/learner.cc:1061: Starting in XGBoost 1.3.0, the default evalu
ation metric used with the objective 'binary:logistic' was changed from
'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore
the old behavior.
```

Classification report 2

In [76]:
```
print(metrics.classification_report(y_valid, tree_yhat))
```

```
              precision    recall  f1-score   support

           0       0.97      0.79      0.87     11821
           1       0.24      0.73      0.36      1056

    accuracy                           0.79     12877
   macro avg       0.60      0.76      0.62     12877
weighted avg       0.91      0.79      0.83     12877
```

In [77]:
```
print(metrics.classification_report(y_valid, knn_yhat))
```

```
              precision    recall  f1-score   support

           0       0.95      0.92      0.93     11821
           1       0.32      0.42      0.36      1056

    accuracy                           0.88     12877
   macro avg       0.63      0.67      0.65     12877
weighted avg       0.90      0.88      0.89     12877
```

In [78]:
```
print(metrics.classification_report(y_valid, lr_yhat))
```

```
              precision    recall  f1-score   support

           0       0.97      0.85      0.91     11821
           1       0.29      0.68      0.41      1056

    accuracy                           0.84     12877
   macro avg       0.63      0.77      0.66     12877
weighted avg       0.91      0.84      0.86     12877
```

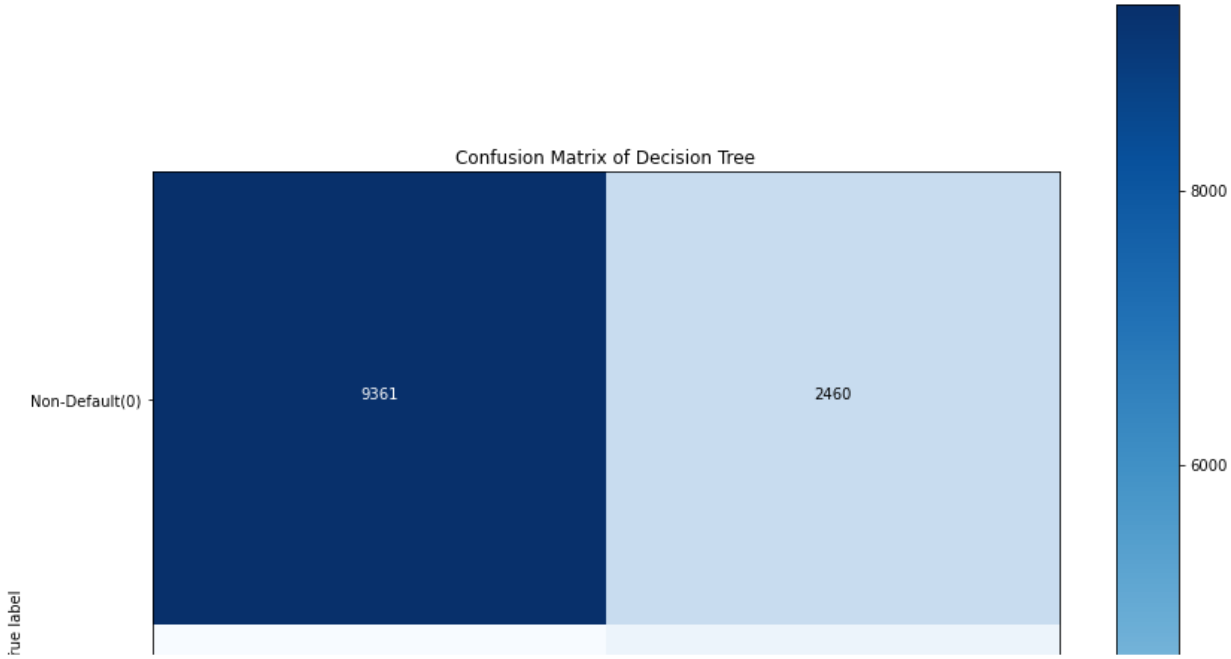
In [79]:
```
print(metrics.classification_report(y_valid, xgb_yhat))
```

```
              precision    recall  f1-score   support

           0       0.96      0.89      0.92     11821
           1       0.30      0.55      0.39      1056

    accuracy                           0.86     12877
   macro avg       0.63      0.72      0.65     12877
weighted avg       0.90      0.86      0.88     12877
```

In [80]:

```
#Confusion Matrix

# defining the plot function

def plot_confusion_matrix(cm, classes, title, normalize = False, cmap = plt
    title = 'Confusion Matrix of {}'.format(title)
    if normalize:
        cm = cm.astype(float) / cm.sum(axis=1)[:, np.newaxis]

    plt.figure(figsize=(6*2, 6*2))    # 8, 6
    plt.imshow(cm, interpolation='nearest', cmap=cmap)
    plt.title(title)
    plt.colorbar()
    tick_marks = np.arange(len(classes))
    plt.xticks(tick_marks, classes, rotation = 45)
    plt.yticks(tick_marks, classes)

    fmt = '.2f' if normalize else 'd'
    thresh = cm.max() / 2.
    for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
        plt.text(j, i, format(cm[i, j], fmt),
                 horizontalalignment = 'center',
                 color = 'white' if cm[i, j] > thresh else 'black')

    plt.tight_layout()
    plt.ylabel('True label')
    plt.xlabel('Predicted label')

# Compute confusion matrix for the models

tree_matrix = confusion_matrix(y_valid, tree_yhat, labels = [0, 1]) # Decis
knn_matrix = confusion_matrix(y_valid, knn_yhat, labels = [0, 1]) # K-Neare
xgb_matrix = confusion_matrix(y_valid, xgb_yhat, labels = [0, 1]) # XGBoost
lr_matrix = confusion_matrix(y_valid, lr_yhat, labels = [0, 1]) # Logistic

# Plot the confusion matrix
plt.rcParams['figure.figsize'] = (6, 6)

# 1. Decision tree

tree_cm_plot = plot_confusion_matrix(tree_matrix,
                                     classes = ['Non-Default(0)','Default(1)'],
                                     normalize = False, title = 'Decision Tree')
plt.savefig('tree_cm_plot.png')
plt.show()

# 2. K-Nearest Neighbors

knn_cm_plot = plot_confusion_matrix(knn_matrix,
                                     classes = ['Non-Default(0)','Default(1)'],
                                     normalize = False, title = 'KNN')
plt.savefig('knn_cm_plot.png')
plt.show()

# 3. XGBoost
```

```
xgb_cm_plot = plot_confusion_matrix(xgb_matrix,
                                    classes = ['Non-Default(0)','Default(1)'],
                                    normalize = False, title = 'XGBoost')
plt.savefig('xgb_cm_plot.png')


# 4. Logistic Regression

lr_cm_plot = plot_confusion_matrix(lr_matrix,
                                    classes = ['Non-Default(0)','Default(1)'],
                                    normalize = False, title = 'LogisticRegress
plt.savefig('lr_cm_plot.png')
```



Predict outcome

In [81]:
```
y_predict = tree_model.predict_proba(imputed_X_test)
death_predict = tree_model.predict(imputed_X_test)

output_tree = pd.DataFrame({'patient_id': test.patient_id,
                            'morality probability':y_predict[:,0],
                            'death probability': y_predict[:, 1],
                            'hospital_death': death_predict}, dtype=np.int3

output_tree.to_csv('submission_Tree.csv', index=False)
output_tree.head()
```

Out[81]:

	patient_id	morality probability	death probability	hospital_death
0	2	0.996875	0.003125	0
1	4	0.613206	0.386794	0
2	5	0.433430	0.566570	1
3	7	0.996875	0.003125	0
4	12	0.613206	0.386794	0

In [82]:
```
y_predict = knn.predict_proba(imputed_X_test)
death_predict = knn.predict(imputed_X_test)

output_knn = pd.DataFrame({'patient_id': test.patient_id,
                           'morality probability': y_predict[:, 0],
                           'death probability': y_predict[:, 1],
                           'hospital_death': death_predict}, dtype=np.int32

output_knn.to_csv('submission_knn.csv', index=False)
output_knn.head()
```

Out[82]:

	patient_id	morality probability	death probability	hospital_death
0	2	1.00	0.00	0
1	4	1.00	0.00	0
2	5	1.00	0.00	0
3	7	1.00	0.00	0
4	12	0.75	0.25	0

In [83]:
```
y_predict = xgb.predict_proba(imputed_X_test)

death_predict = xgb.predict(imputed_X_test)

output_xgb = pd.DataFrame({'patient_id': test.patient_id,
                           'morality probability': y_predict[:, 0],
                           'death probability': y_predict[:, 1],
                           'hospital_death': death_predict}, dtype=np.int32

output_xgb.to_csv('submission_xgb.csv', index=False)
output_xgb.head()
```

Out[83]:

	patient_id	morality probability	death probability	hospital_death
0	2	0.980382	0.019618	0
1	4	0.607508	0.392492	0
2	5	0.848357	0.151643	0
3	7	0.991595	0.008405	0
4	12	0.839526	0.160474	0

In [84]:
```
y_predict = lr.predict_proba(imputed_X_test)
death_predict = lr.predict(imputed_X_test)

output_lr = pd.DataFrame({'patient_id': test.patient_id,
                          'morality probability': y_predict[:, 0],
                          'death probability': y_predict[:, 1],
                          'hospital_death': death_predict}, dtype=np.int32

output_lr.to_csv('submission_lr.csv', index=False)
output_lr.head()
```

Out[84]:

	patient_id	morality probability	death probability	hospital_death
0	2	0.809561	0.190439	0
1	4	0.442449	0.557551	1
2	5	0.721127	0.278873	0
3	7	0.805889	0.194111	0
4	12	0.637025	0.362975	0

In [ ]: